Q-Learning

Kianté Brantley
CS 1840 Fall 2026

Announcements

HW2 is out and is due 10/8 (this Thursday)!

HW3 will be released on Friday!

Check the course website for Office Hours:
https://harvard-cs-1840.github.io/2026/

Finishing Up From Last Lecture



1. Simple Monte Carlo methods

2. Temporal Difference Learning

Recap: MC estimation with incremental update

Instead of waiting for all N trajs, we can also update the estimator incrementally every traj

Initialize $\hat{V}^{\pi}(s) = 0, \forall s$

For all state $s \in \mathcal{S}$:

Recap: MC estimation with incremental update

Instead of waiting for all N trajs, we can also update the estimator incrementally every traj

Initialize $\hat{V}^{\pi}(s) = 0, \forall s$

For all state $s \in \mathcal{S}$:

Generate one traj from $s_0 = s$ using π;

Recap: MC estimation with incremental update

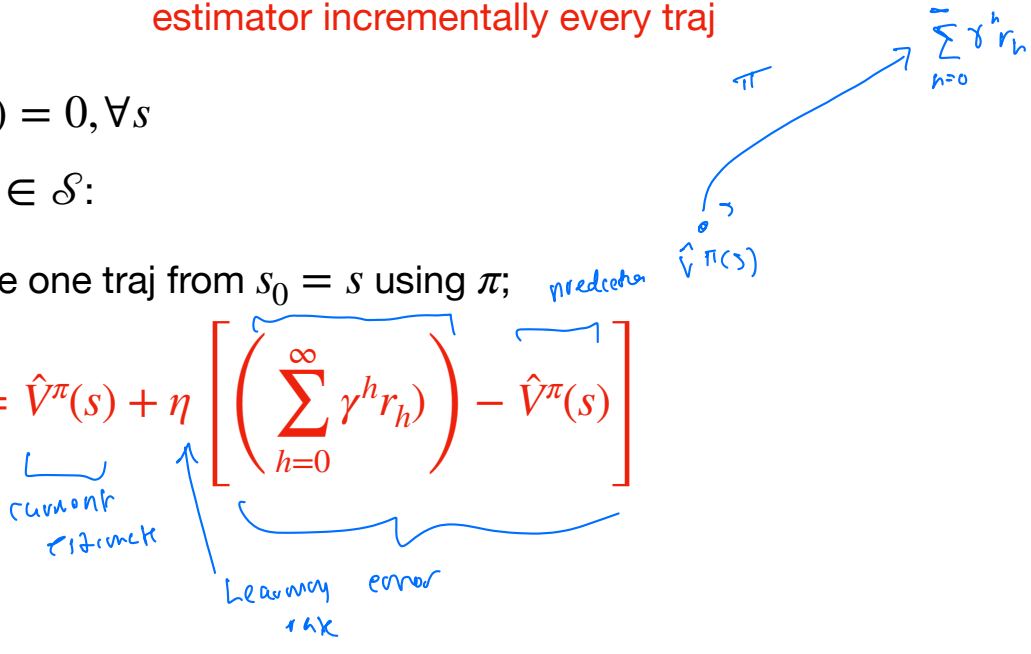
Instead of waiting for all N trajs, we can also update the estimator incrementally every traj

Initialize $\hat{V}^{\pi}(s) = 0, \forall s$

For all state $s \in \mathcal{S}$:

Generate one traj from $s_0 = s$ using π;

$$\hat{V}^{\pi}(s) \Leftarrow \hat{V}^{\pi}(s) + \eta \left[\left(\sum_{h=0}^{\infty} \gamma^h r_h \right) - \hat{V}^{\pi}(s) \right]$$



Recap: MC estimation with incremental update

Instead of waiting for all N trajs, we can also update the estimator incrementally every traj

Initialize $\hat{V}^{\pi}(s) = 0, \forall s$

For all state $s \in \mathcal{S}$:

Generate one traj from $s_0 = s$ using π;

$$\hat{V}^{\pi}(s) \Leftarrow \hat{V}^{\pi}(s) + \eta \left[\left(\sum_{h=0}^{\infty} \gamma^h r_h \right) - \hat{V}^{\pi}(s) \right]$$

total reward of one traj

Recap: MC estimation with incremental update

Instead of waiting for all N trajs, we can also update the estimator incrementally every traj

Initialize $\hat{V}^{\pi}(s) = 0, \forall s$

For all state $s \in \mathcal{S}$:

Generate one traj from $s_0 = s$ using π;

$$\hat{V}^{\pi}(s) \Leftarrow \hat{V}^{\pi}(s) + \eta \left[\left(\sum_{h=0}^{\infty} \gamma^h r_h \right) - \hat{V}^{\pi}(s) \right]$$

η: learning rate

total reward of one traj

Recap: TD Learning

Initialize $V^{\pi}(s) = 0, \forall s$. Set initial state $s \in \mathcal{S}$

While True:

Recap: TD Learning

Initialize $V^\pi(s) = 0, \forall s$. Set initial state $s \in \mathcal{S}$

While True:

$a \sim \pi(\cdot \mid s)$
$r = r(s, a)$
$s' \sim p(\cdot \mid s, a)$

> Take action $a \sim \pi(\cdot \mid s)$, get reward r and next state $s' \sim P(\cdot \mid s, a)$

Recap: TD Learning

Initialize $V^{\pi}(s) = 0, \forall s$. Set initial state $s \in \mathcal{S}$

While True:

> Take action $a \sim \pi(\cdot \mid s)$, get reward r and next state $s' \sim P(\cdot \mid s, a)$
>
> Form TD target $r + \gamma \hat{V}^{\pi}(s')$

Recap: TD Learning

Initialize $V^{\pi}(s) = 0, \forall s$. Set initial state $s \in \mathcal{S}$

While True:

Take action $a \sim \pi(\cdot \mid s)$, get reward r and next state $s' \sim P(\cdot \mid s, a)$

Form TD target $r + \gamma \hat{V}^{\pi}(s')$

Update for s: $\hat{V}^{\pi}(s) \Leftarrow \hat{V}^{\pi}(s) + \eta \left(r + \gamma \hat{V}^{\pi}(s') - \hat{V}^{\pi}(s) \right)$

TD-target

current estimate

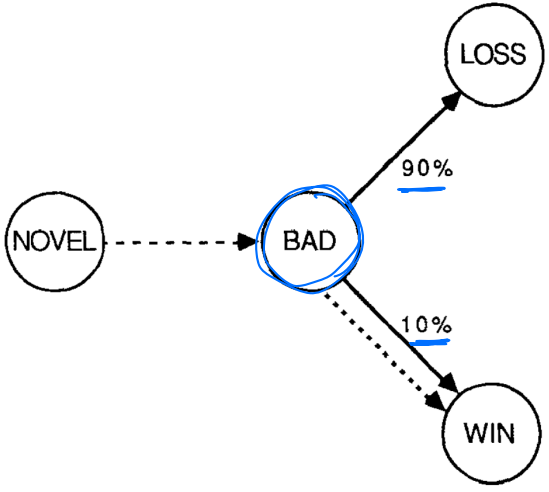
Recap: TD Learning

Initialize $V^\pi(s) = 0, \forall s$. Set initial state $s \in \mathcal{S}$

While True:

> Take action $a \sim \pi(\cdot \mid s)$, get reward r and next state $s' \sim P(\cdot \mid s, a)$
>
> Form TD target $r + \gamma \hat{V}^\pi(s')$
>
> Update for s: $\hat{V}^\pi(s) \Leftarrow \hat{V}^\pi(s) + \eta \left(r + \gamma \hat{V}^\pi(s') - \hat{V}^\pi(s) \right)$
>
> Set $s \Leftarrow s'$

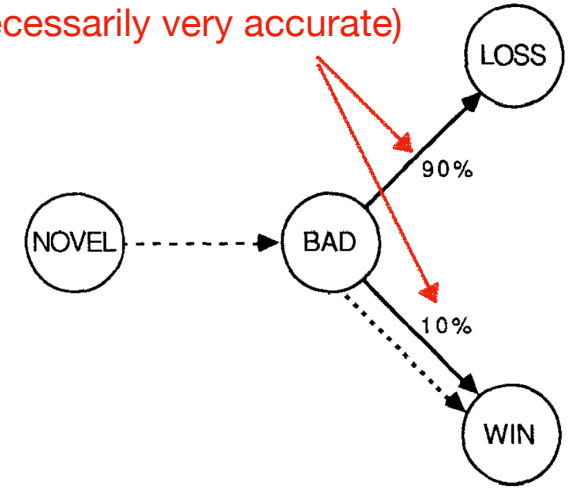
Example of faster learning with TD



[Sutton, 1988]

Example of faster learning with TD

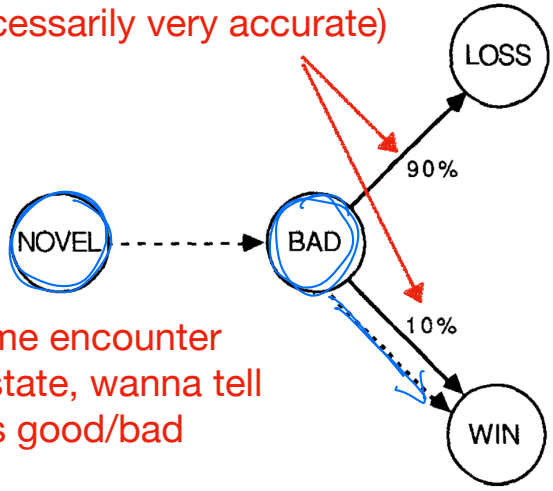
From past game experiences
(not necessarily very accurate)



[Sutton, 1988]

Example of faster learning with TD

From past game experiences
(not necessarily very accurate)

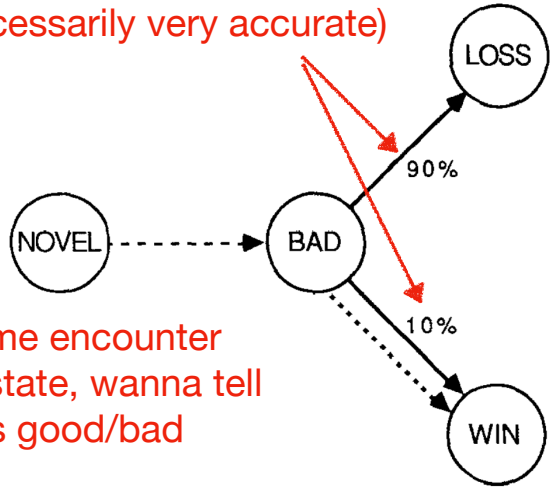


first time encounter
“novel” state, wanna tell
if it’s good/bad

[Sutton, 1988]

Example of faster learning with TD

From past game experiences
(not necessarily very accurate)



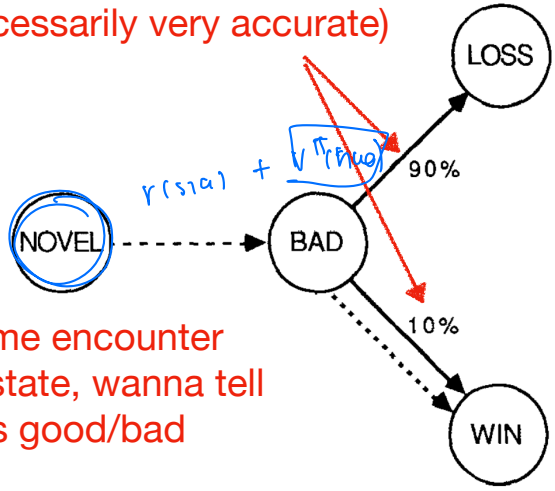
first time encounter
“novel” state, wanna tell
if it’s good/bad

[Sutton, 1988]

MC: have to rollout from NOVEL many times
to get a reasonable estimate of future

Example of faster learning with TD

From past game experiences
(not necessarily very accurate)



first time encounter
“novel” state, wanna tell
if it’s good/bad

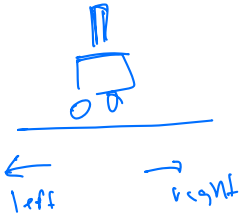
[Sutton, 1988]

MC: have to rollout from NOVEL many times to get a reasonable estimate of future

TD (bootstrapping): NOVEL leading to BAD in one-step implies NOVEL is likely bad

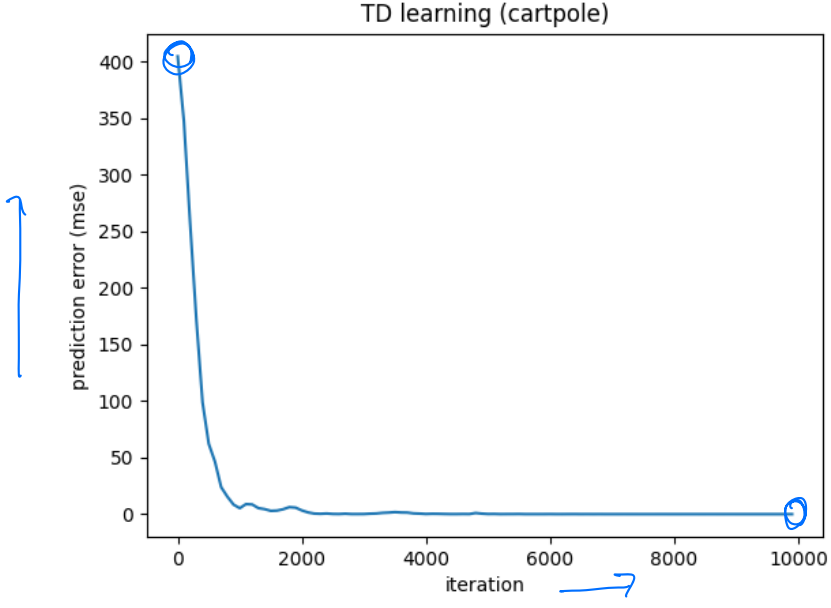
TD learning on CartPole

Note: Cartpole's state is continuous, so we will need TD w/ function approximation, e.g., neural network (we will get there very soon)



TD learning on CartPole

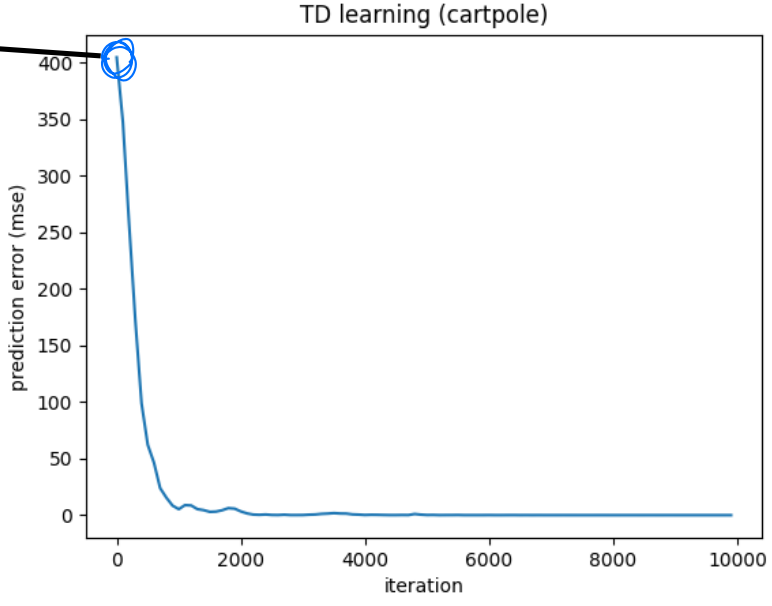
Note: Cartpole's state is continuous, so we will need TD w/ function approximation, e.g., neural network (we will get there very soon)



TD learning on CartPole

Note: Cartpole's state is continuous, so we will need TD w/ function approximation, e.g., neural network (we will get there very soon)

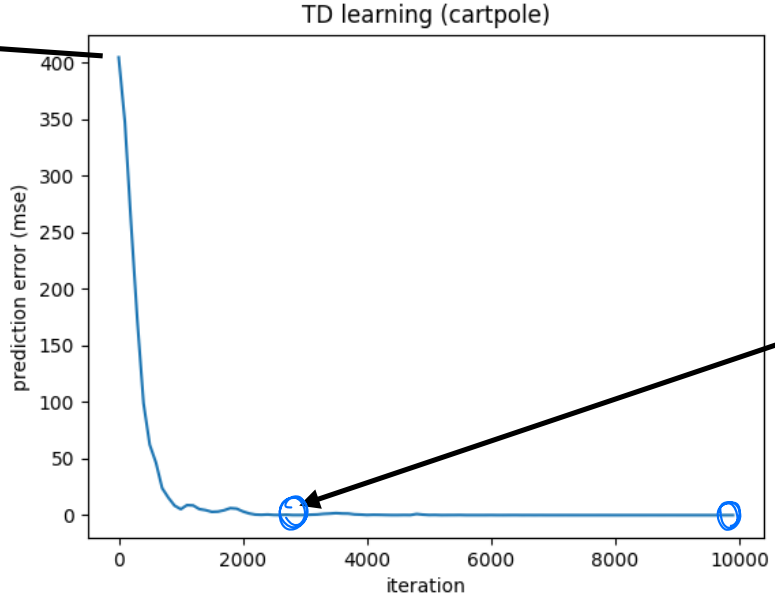
A randomly initialized NN as $\hat{V}^{\pi}(s)$ (very bad estimator)



TD learning on CartPole

Note: Cartpole's state is continuous, so we will need TD w/ function approximation, e.g., neural network (we will get there very soon)

A randomly initialized NN as $\hat{V}^{\pi}(s)$ (very bad estimator)



Almost perfectly estimate the true target $V^{\pi}(s)$

TD vs MC: bias-variance tradeoff

TD vs MC: bias-variance tradeoff

When would we use MC?

TD vs MC: bias-variance tradeoff

When would we use MC?

MC is unbiased, but has higher variance (i.e., randomness over an entire traj)

TD vs MC: bias-variance tradeoff

When would we use MC?

MC is unbiased, but has higher variance (i.e., randomness over an entire traj)

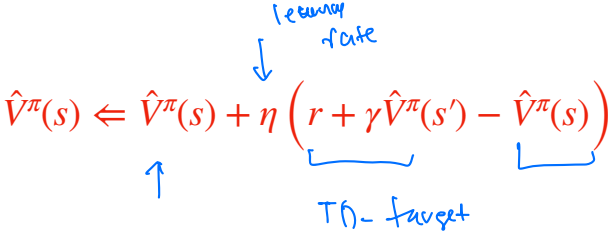
TD can have high bias, but has lower variance

TD vs MC: bias-variance tradeoff

When would we use MC?

MC is unbiased, but has higher variance (i.e., randomness over an entire traj)

TD can have high bias, but has lower variance



TD vs MC: bias-variance tradeoff

When would we use MC?

MC is unbiased, but has higher variance (i.e., randomness over an entire traj)

TD can have high bias, but has lower variance

$$\sum_{n=0}^{\infty} \gamma^n r_n$$

$$\hat{V}^\pi(s) \Leftarrow \hat{V}^\pi(s) + \eta \left(r + \gamma \hat{V}^\pi(s') - \hat{V}^\pi(s) \right)$$

Randomness in target is
from **one-step transition**

TD vs MC: bias-variance tradeoff

When would we use MC?

MC is unbiased, but has higher variance (i.e., randomness over an entire traj)

TD can have high bias, but has lower variance

Large bias when $\hat{V}^\pi(s')$ is far from $V^\pi(s')$

$$\hat{V}^\pi(s) \Leftarrow \hat{V}^\pi(s) + \eta \left(r + \gamma \hat{V}^\pi(s') - \hat{V}^\pi(s) \right)$$

Randomness in target is from **one-step transition**

Recap: Questions

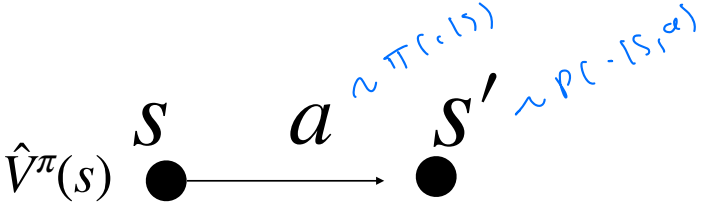
Can we extend the one-step TD target to include multiple future rewards before bootstrapping?

TD relies on the idea of **Bootstrapping** — *using predictions from our estimator to update the estimator itself*

Recap: Questions

Can we extend the one-step TD target to include multiple future rewards before bootstrapping?

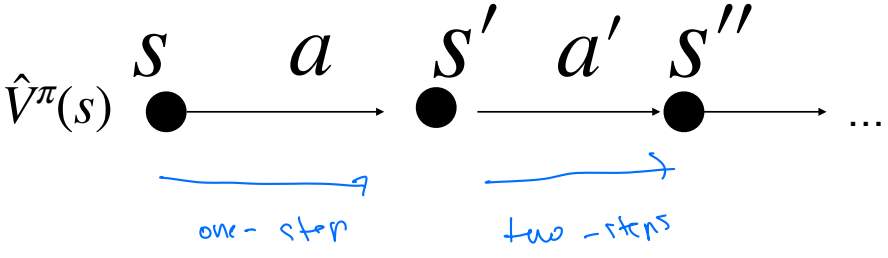
TD relies on the idea of **Bootstrapping** — *using predictions from our estimator to update the estimator itself*



Recap: Questions

Can we extend the one-step TD target to include multiple future rewards before bootstrapping?

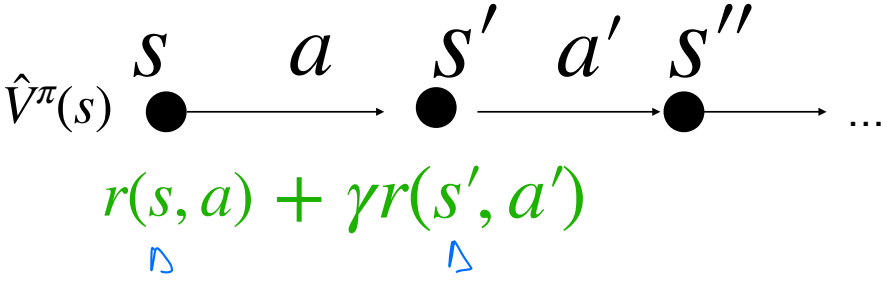
TD relies on the idea of **Bootstrapping** — *using predictions from our estimator to update the estimator itself*



Recap: Questions

Can we extend the one-step TD target to include multiple future rewards before bootstrapping?

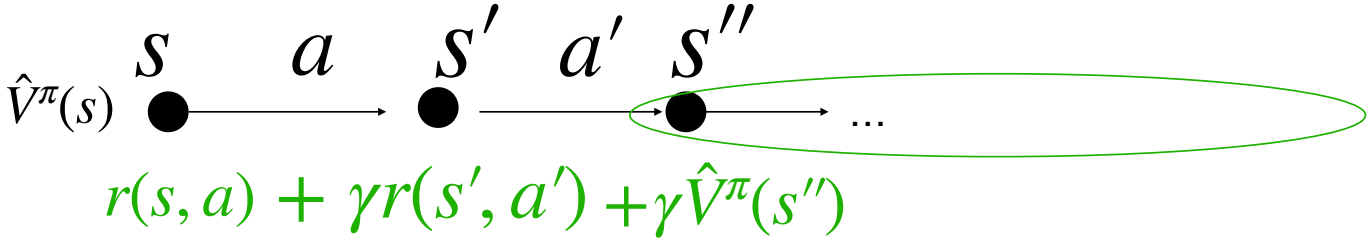
TD relies on the idea of **Bootstrapping** — *using predictions from our estimator to update the estimator itself*



Recap: Questions

Can we extend the one-step TD target to include multiple future rewards before bootstrapping?

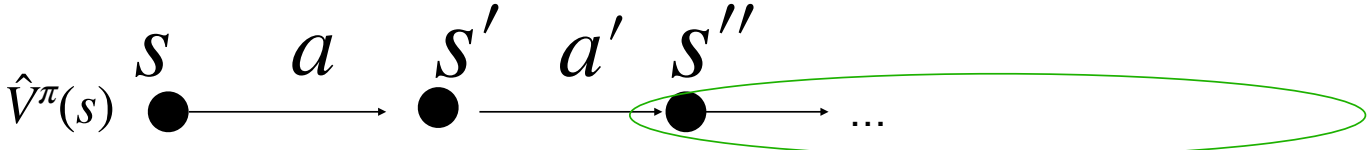
TD relies on the idea of **Bootstrapping** — *using predictions from our estimator to update the estimator itself*



Recap: Questions

Can we extend the one-step TD target to include multiple future rewards before bootstrapping?

TD relies on the idea of **Bootstrapping** — *using predictions from our estimator to update the estimator itself*



$$r(s,a) + \gamma r(s',a') + \gamma \hat{V}^{\pi}(s'')$$

New target value (bootstrapping)

Recap: Bellman Optimality

Bellman Optimality

$$Q^\star(s,a) = r(s,a) + \gamma \mathbb{E}_{s' \sim P(\cdot|s,a)} \max_{a'} Q^\star(s',a'), \forall s, a$$

VI: An iterative approach for estimating $Q^\star$

$$Q \Leftarrow \mathcal{T} Q$$

$(\mathcal{T}Q)(s,a) = r(s,a) + \gamma \mathbb{E}_{s' \sim P(s,a)} \left[\max_{a'} Q(s,a) \right]$

Recap: Bellman Optimality

Bellman Optimality

$$Q^\star(s,a) = r(s,a) + \gamma \mathbb{E}_{s' \sim P(\cdot|s,a)} \max_{a'} Q^\star(s',a'), \forall s, a$$

VI: An iterative approach for estimating $Q^\star$

$$Q \Leftarrow \mathcal{T} Q$$

1. Need to know the transition
2. Only works for discrete small MDPs

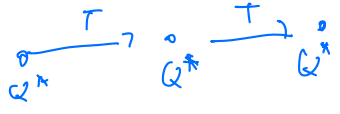
Recap: Bellman Optimality

Q: if there is some $Q(s,a)$, such that the following holds:

$$Q(s,a) = r(s,a) + \gamma \mathbb{E}_{s' \sim P(\cdot|s,a)} \max_{a'} Q(s',a'), \forall s, a$$

is this $Q = Q^\star$?

$Q^* = TQ^*$



Today

Given MDP $\mathcal{M} = (S, A, r, P, \gamma)$,

how to estimate $Q^\star(s, a), \forall s$ WITHOUT knowing P
i.e., how to learn $Q^\star$ (thus, $\pi^\star$) from experience

$\pi^* = \underset{a}{\operatorname{argmax}} \; Q^*(s, a)$

Motivation

Computing a near-optimal policy to achieve the long-term goals w/o knowing or explicitly modeling the world





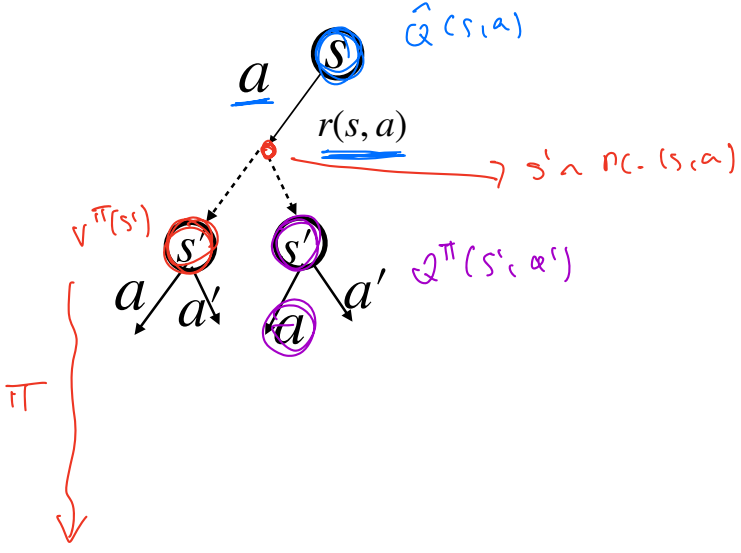


Outline:

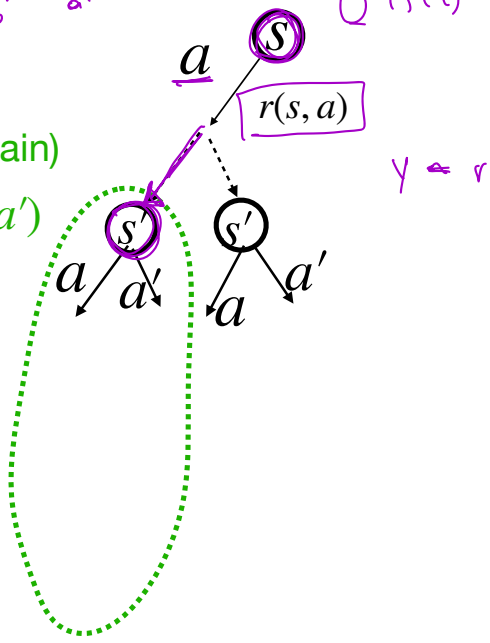
1. Q Learning

2. Revisit TD: Off-policy TD Learning

Q Learning



Q Learning

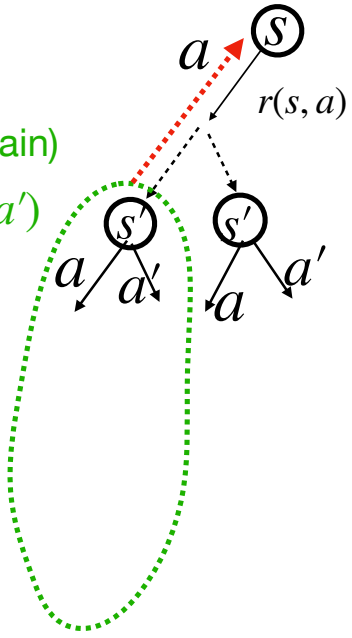


(Bootstrapping again)

$$\max_{a'} \hat{Q}(s', a')$$

Q Learning

$$Q^\star(s,a) \approx \underbrace{r(s,a) + \gamma \max_{a'} \hat{Q}(s',a')}_{\text{target}}$$

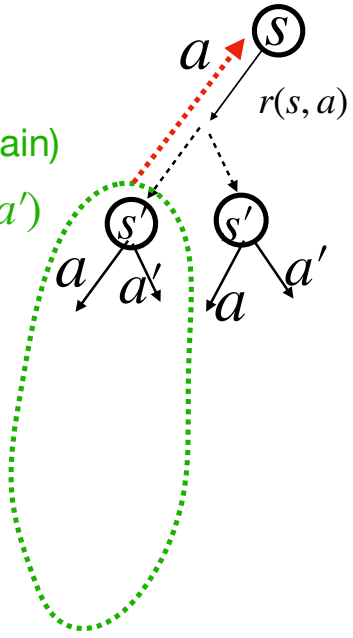


(Bootstrapping again)

$$\max_{a'} \hat{Q}(s',a')$$

Q Learning

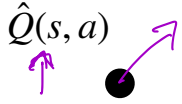
$$Q^\star(s,a) \approx \underbrace{r(s,a) + \gamma \max_{a'} \hat{Q}(s',a')}_{\text{target}}$$



(Bootstrapping again)

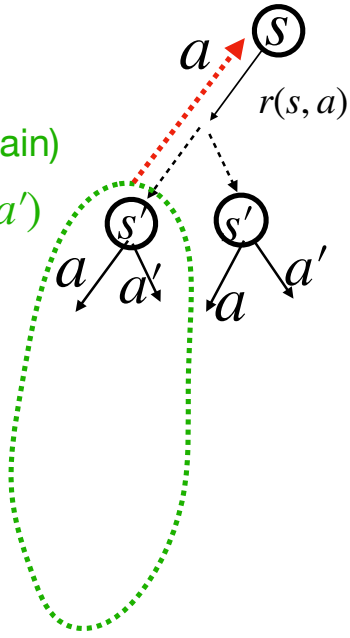
$$\max_{a'} \hat{Q}(s',a')$$

Target



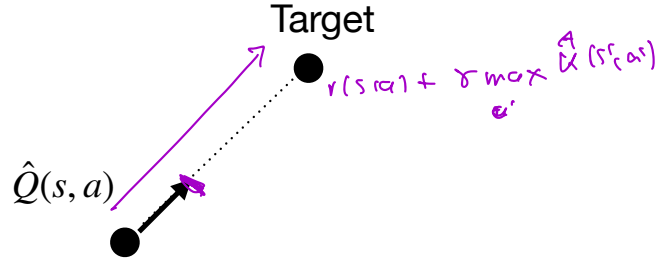
Q Learning

$$Q^\star(s,a) \approx \underbrace{r(s,a) + \gamma \max_{a'} \hat{Q}(s',a')}_{\text{target}}$$



(Bootstrapping again)

$$\max_{a'} \hat{Q}(s',a')$$



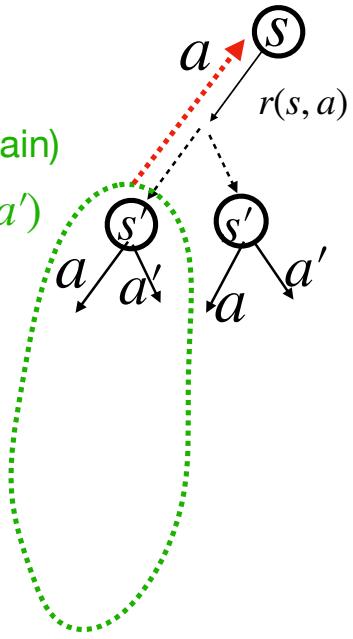
Q Learning

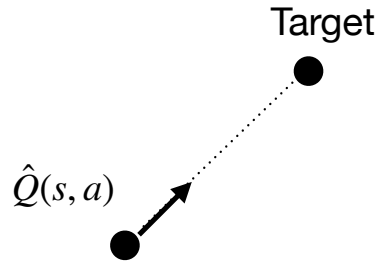
$$Q^\star(s,a) \approx r(s,a) + \gamma \max_{a'} \hat{Q}(s',a')$$

target

(Bootstrapping again)

$$\max_{a'} \hat{Q}(s',a')$$





Q-learning update: move to the target with a small step

Q Learning

Given a **one-step transition** (s, a, r, s') where $r = r(s, a), s' \sim P(\cdot \mid s, a)$:

Q-learning updates the guess at (s, a) as follows:

$$\hat{Q}(s,a) \Leftarrow \hat{Q}(s,a) + \eta \left(r + \gamma \max_{a'} \hat{Q}(s', a') - \hat{Q}(s,a) \right)$$

Target

Q-Target
(Constructed via Bootstrapping!)

$Q^* \rightarrow \pi^* = \arg\max_a Q^*(s, a)$

Q Learning

How to collect data?

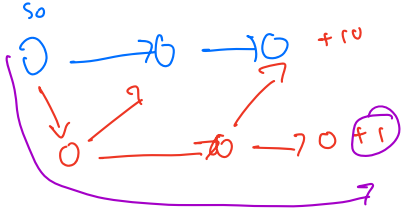
Choice one: trust current estimator $\hat{Q}$, always use $\arg\max_a \hat{Q}(s, a)$

Q Learning

How to collect data?

Choice one: trust current estimator $\hat{Q}$, always use $\arg\max_a \hat{Q}(s, a)$

Issue: cannot explore (i.e., need to try something that hasn't been tried)



Q Learning

How to collect data?

Choice one: trust current estimator $\hat{Q}$, always use $\arg\max_a \hat{Q}(s, a)$

Issue: cannot explore (i.e., need to try something that hasn't been tried)

Choice two (quite effective in practice): ϵ**-greedy**

$\epsilon \in (0, 1)$

Q Learning

How to collect data?

Choice one: trust current estimator $\hat{Q}$, always use $\arg\max_a \hat{Q}(s,a)$

Issue: cannot explore (i.e., need to try something that hasn't been tried)

Choice two (quite effective in practice): ϵ**-greedy**

W/ prob ϵ, select action uniform randomly ← Explore

W/ prob $1-\epsilon$, select greedy action $\arg\max_a \hat{Q}(s,a)$ ← Exploit

Choice three (softmax exploration)

$\hat{\pi}(\cdot|s) = \text{softmax}(\hat{Q}) = \frac{\exp(\hat{Q}(s,a)/\tau)}{\sum_{a'} \exp(\hat{Q}(s,a')/\tau)}$ then $a \sim \hat{\pi}$

Q Learning

Initialize $\hat{Q}(s, a) = 0, \forall s, a$. Set initial state $s \in \mathcal{S}$

While True:

Q Learning

Initialize $\hat{Q}(s,a) = 0, \forall s, a$. Set initial state $s \in \mathcal{S}$

While True:

> Take action a based on ϵ-greedy of $\hat{Q}$, get reward r and next state $s' \sim P(\cdot \mid s, a)$

Q Learning

Initialize $\hat{Q}(s, a) = 0, \forall s, a$. Set initial state $s \in \mathcal{S}$

While True:

> Take action a based on ϵ-greedy of $\hat{Q}$, get reward r and next state $s' \sim P(\cdot \mid s, a)$
>
> Form Q-target $r + \gamma \max_{a'} \hat{Q}(s', a')$

Q Learning

Initialize $\hat{Q}(s,a) = 0, \forall s, a$. Set initial state $s \in \mathcal{S}$

While True:

> Take action a based on ϵ-greedy of $\hat{Q}$, get reward r and next state $s' \sim P(\cdot \mid s, a)$
>
> Form Q-target $r + \gamma \max_{a'} \hat{Q}(s', a')$
>
> Update for s, a: $\hat{Q}(s,a) \Leftarrow \hat{Q}(s,a) + \eta \left(r + \gamma \max_{a'} \hat{Q}(s', a') - \hat{Q}(s,a) \right)$

error

Q-target

current estimate

Q Learning

Initialize $\hat{Q}(s,a) = 0, \forall s, a$. Set initial state $s \in \mathcal{S}$

While True:

> Take action a based on ϵ-greedy of $\hat{Q}$, get reward r and next state $s' \sim P(\cdot \mid s, a)$
>
> Form Q-target $r + \gamma \max_{a'} \hat{Q}(s', a')$
>
> Update for s, a: $\hat{Q}(s,a) \Leftarrow \hat{Q}(s,a) + \eta \left(r + \gamma \max_{a'} \hat{Q}(s', a') - \hat{Q}(s,a) \right)$
>
> Set $s \Leftarrow s'$

Interpret Q-learning as “SGD” on Bellman error

Q-learning is not the usual SGD, i.e., it is **not** running SGD to minimize a fixed loss function

Q-learning may be interpreted as running SGD on an **evolving** loss function (Bellman error)

Interpret Q-learning as "SGD" on Bellman error

Q-learning is not the usual SGD, i.e., it is **not** running SGD to minimize a fixed loss function

Q-learning may be interpreted as running SGD on an **evolving** loss function (Bellman error)

$$\ell_{be}(\hat{Q}(s,a)) := \left(\hat{Q}(s,a) - y\right)^2, \text{ where } y = r(s,a) + \gamma \mathbb{E}_{s' \sim P(\cdot|s,a)} \max_{a'} \hat{Q}(s',a')$$

$\hat{Q} = T\hat{Q}$

Bootstrap

Interpret Q-learning as “SGD” on Bellman error

Q-learning is not the usual SGD, i.e., it is **not** running SGD to minimize a fixed loss function

Q-learning may be interpreted as running SGD on an **evolving** loss function (Bellman error)

$$\ell_{be}(\hat{Q}(s,a)) := \left(\hat{Q}(s,a) - y\right)^2, \text{ where } y = r(s,a) + \gamma \mathbb{E}_{s' \sim P(\cdot|s,a)} \max_{a'} \hat{Q}(s',a')$$

This keeps changing as we learning

Interpret Q-learning as "SGD" on Bellman error

Q-learning is not the usual SGD, i.e., it is **not** running SGD to minimize a fixed loss function

Q-learning may be interpreted as running SGD on an **evolving** loss function (Bellman error)

$$\ell_{be}(\hat{Q}(s,a)) := \left(\hat{Q}(s,a) - y\right)^2, \text{ where } y = r(s,a) + \gamma \mathbb{E}_{s' \sim P(\cdot|s,a)} \max_{a'} \hat{Q}(s',a')$$

This keeps changing as we learning

$$\nabla \ell_{be}(x)|_{x=\hat{Q}(s,a)} := 2\left(\hat{Q}(s,a) - y\right)$$

$s' \sim P(\cdot|s,a)$

$r(s,a) + \gamma \max_{a'} \hat{Q}(s',a')$

Interpret Q-learning as "SGD" on Bellman error

Q-learning is not the usual SGD, i.e., it is **not** running SGD to minimize a fixed loss function

Q-learning may be interpreted as running SGD on an **evolving** loss function (Bellman error)

$$\ell_{be}(\hat{Q}(s,a)) := \left(\hat{Q}(s,a) - y\right)^2, \text{ where } y = r(s,a) + \gamma \mathbb{E}_{s' \sim P(\cdot|s,a)} \max_{a'} \hat{Q}(s',a')$$

This keeps changing as we learning

$$\nabla \ell_{be}(x)|_{x=\hat{Q}(s,a)} := 2\left(\hat{Q}(s,a) - y\right)$$

$$\widetilde{\nabla} \ell_{be}(x)|_{x=\hat{Q}(s,a)} := 2\left(\hat{Q}(s,a) - \left(r + \gamma \max_{a'} \hat{Q}(s',a')\right)\right)$$

Interpret Q-learning as "SGD" on Bellman error

Q-learning is not the usual SGD, i.e., it is **not** running SGD to minimize a fixed loss function

Q-learning may be interpreted as running SGD on an **evolving** loss function (Bellman error)

$$\ell_{be}(\hat{Q}(s,a)) := \left(\hat{Q}(s,a) - y\right)^2, \text{ where } y = r(s,a) + \gamma \mathbb{E}_{s' \sim P(\cdot|s,a)} \max_{a'} \hat{Q}(s',a')$$

$$\nabla \ell_{be}(x)|_{x=\hat{Q}(s,a)} := 2\left(\hat{Q}(s,a) - y\right)$$

Unbiased estimate of y

This keeps changing as we learning

$$\widetilde{\nabla} \ell_{be}(x)|_{x=\hat{Q}(s,a)} := 2\left(\hat{Q}(s,a) - \left(r + \gamma \max_{a'} \hat{Q}(s',a')\right)\right)$$

$$\hat{Q}(s,a) \leftarrow \hat{Q}(s,a) - \eta \, \widetilde{\nabla} \ell_{be}(x)|_{x=\hat{Q}(s,a)}$$

Q Learning Theory

[Informal] Assume the ϵ-greedy strategy has non-trivial probability of visiting every state-action pair. Setting learning rate η properly, we will have:

$$\hat{Q}(s,a) \rightarrow Q^{\star}(s,a), \forall s, a$$

when # of interactions approaches to ∞

(concrete convergence rates are known as well)

Outline:



1. Q Learning

2. Revisit TD: Off-policy TD Learning

TD Learning

Given (s, a, r, s'), where $a \sim \pi(\cdot \mid s), s' \sim P(\cdot \mid s, a)$, TD updates:

$$\hat{V}^{\pi}(s) \Leftarrow \hat{V}^{\pi}(s) + \eta \left(r + \gamma \hat{V}^{\pi}(s') - \hat{V}^{\pi}(s') \right)$$

TD-target

On-policy: data is generated from the policy π itself

Off-policy: data is generated from policy π_b where $\pi_b \neq \pi$

TD Learning

Given (s, a, r, s'), where $a \sim \pi(\cdot \mid s), s' \sim P(\cdot \mid s, a)$, TD updates:

$$\hat{V}^{\pi}(s) \Leftarrow \hat{V}^{\pi}(s) + \eta \left(r + \gamma \hat{V}^{\pi}(s') - \hat{V}^{\pi}(s') \right)$$

On-policy: data is generated from the policy π itself

Off-policy: data is generated from policy π_b where $\pi_b \neq \pi$

Q: is Q-learning off-policy or on-policy?

$\underset{a}{\operatorname{argmax}}\ \hat{Q}(s, a)$

Motivation for off-policy evaluation

Counterfactual: what would happen if I did something different?

Off-policy TD Learning

Setting: data is generated by π_b, but we want to estimate V^π for some $\pi \neq \pi_b$

Key trick: importance weighting

$$V^\pi(s) = \mathbb{E}_{a\sim\pi(\cdot|s)}\left(r(s,a) + \gamma \mathbb{E}_{s'\sim P(\cdot|s,a)} V^\pi(s')\right)$$

Off-policy TD Learning

Setting: data is generated by π_b, but we want to estimate V^π for some $\pi \neq \pi_b$

Key trick: importance weighting

$$V^\pi(s) = \mathbb{E}_{a\sim\pi(\cdot|s)}\left(r(s,a) + \gamma \mathbb{E}_{s'\sim P(\cdot|s,a)} V^\pi(s')\right)$$

$$= \mathbb{E}_{a\sim\pi_b(\cdot|s)} \frac{\pi(a\,|\,s)}{\pi_b(a\,|\,s)}\left(r(s,a) + \gamma \mathbb{E}_{s'\sim P(\cdot|s,a)} V^\pi(s')\right)$$

Off-policy TD Learning

Setting: data is generated by π_b, but we want to estimate V^π for some $\pi \neq \pi_b$

Key trick: importance weighting

$$V^\pi(s) = \mathbb{E}_{a\sim\pi(\cdot|s)}\left(r(s,a) + \gamma \mathbb{E}_{s'\sim P(\cdot|s,a)} V^\pi(s')\right)$$

$$= \mathbb{E}_{a\sim\pi_b(\cdot|s)} \frac{\pi(a\,|\,s)}{\pi_b(a\,|\,s)} \left(r(s,a) + \gamma \mathbb{E}_{s'\sim P(\cdot|s,a)} V^\pi(s')\right)$$

Importance weight

Off-policy TD Learning

Setting: data is generated by π_b, but we want to estimate V^π for some $\pi \neq \pi_b$

Key trick: importance weighting

$$V^\pi(s) = \mathbb{E}_{a\sim\pi(\cdot|s)}\left(r(s,a) + \gamma \mathbb{E}_{s'\sim P(\cdot|s,a)} V^\pi(s')\right)$$

$$= \mathbb{E}_{a\sim\pi_b(\cdot|s)} \frac{\pi(a\,|\,s)}{\pi_b(a\,|\,s)} \left(r(s,a) + \gamma \mathbb{E}_{s'\sim P(\cdot|s,a)} V^\pi(s')\right)$$

Now action is sampled from π_b

Importance weight

Off-policy TD Learning

Given (s, a, r, s'), where $a \sim \pi_b(\cdot \mid s), s' \sim P(\cdot \mid s, a)$,

Off-policy TD updates as follows:

$$\hat{V}^{\pi}(s) \Leftarrow \hat{V}^{\pi}(s) + \eta \frac{\pi(a \mid s)}{\pi_b(a \mid s)} \left(r + \gamma \hat{V}^{\pi}(s') - V^{\pi}(s) \right)$$

Off-policy TD Learning

Given (s, a, r, s'), where $a \sim \pi_b(\cdot \mid s), s' \sim P(\cdot \mid s, a)$,

Off-policy TD updates as follows:

$$\hat{V}^{\pi}(s) \Leftarrow \hat{V}^{\pi}(s) + \eta \frac{\pi(a \mid s)}{\pi_b(a \mid s)} \left(r + \gamma \hat{V}^{\pi}(s') - V^{\pi}(s) \right)$$

Case 1: $\pi(a \mid s)$ is large but $\pi_b(a \mid s)$ is small

Off-policy TD Learning

Given (s, a, r, s'), where $a \sim \pi_b(\cdot \mid s), s' \sim P(\cdot \mid s, a)$,

Off-policy TD updates as follows:

$$\hat{V}^{\pi}(s) \Leftarrow \hat{V}^{\pi}(s) + \eta \frac{\pi(a \mid s)}{\pi_b(a \mid s)} \left(r + \gamma \hat{V}^{\pi}(s') - V^{\pi}(s) \right)$$

Case 1: $\pi(a \mid s)$ is large but $\pi_b(a \mid s)$ is small

Case 2: $\pi(a \mid s)$ is small but $\pi_b(a \mid s)$ is large

Summary

Q-Learning: online algorithm that learns $Q^\star$ (bootstrapping)

Exploration & Exploitation tradeoff: ϵ-greedy is an effective heuristic

Off-policy policy evaluation: importance weighting
(also known as inverse probability weighting in causal inference)

Feedback and Attendance

https://bit.ly/4gYn2nC

